

Cyclic Oriented Graph Counting

Complete Guided Notes for Your Project

March 5, 2026

Abstract

This note explains the cyclic counting part of your project in a proof-style but beginner-friendly way. It defines every object, every symmetry, and every formula used by the code path `count_adjacency_burnside(n)` in `cyclic_sequences.py`. It also includes a full worked example for $n = 8$, matching the exact computed values 288, 64, and final answer 22.

Contents

1	What Problem Is Being Solved?	2
2	Core Objects and Definitions	2
2.1	Cycle and indexing	2
2.2	Vertex signature	2
2.3	Adjacency constraint (project default fast path)	3
3	Symmetry: Why We Need Group Actions	3
4	Canonical Representative	3
5	Burnside Lemma: The Orbit Counter	4
5.1	Statement	4
5.2	What “fixed” means	4
5.3	Why this is correct (double-counting sketch)	4
6	Why Burnside Alone Is Not Enough	4
7	Transfer Matrix for Adjacency-valid Cycles	4
7.1	State space used in your code	4
7.2	One-step update	5
7.3	Matrix construction	5
7.4	Closed walks and trace	5
8	Burnside Rotation Term in This Project	5

9 Burnside Reflection Term in This Project	6
10 Final Counting Formula in Code	6
11 Worked Example: $n = 8$ (Exact Values From Your Program)	6
12 Optional Constraints in the Cyclic Module	7
12.1 Primitivity	7
12.2 Forbidden subsequence mode	7
13 Validation and Correctness Strategy	7
14 Complexity Intuition	7
15 Presentation-ready 90-second summary	8
16 Glossary of Symbols	8
17 Takeaway	8

1 What Problem Is Being Solved?

We study directed cycles with n edges. Each edge gets one of two orientations:

- bit 0: edge points from vertex i to $i + 1 \pmod n$;
- bit 1: edge points from vertex $i + 1$ to i .

So each object is a binary sequence $(e_0, \dots, e_{n-1}) \in \{0, 1\}^n$.

The target is not the raw number 2^n . We count *distinct objects up to symmetry* (rotations and reflections). Optionally, we enforce constraints such as adjacency and primitivity.

2 Core Objects and Definitions

2.1 Cycle and indexing

Vertices are $V = \{0, 1, \dots, n - 1\}$, arranged cyclically. Edge i connects vertex i and vertex $i + 1 \pmod n$.

2.2 Vertex signature

At each vertex v , two incident edges determine a signature in $\{-2, 0, +2\}$:

$$\sigma(v) = 2 \cdot (\text{number of incoming incident edges at } v) - 2.$$

So:

- -2 : both incident edges point out of v ,
- 0 : one in and one out,
- $+2$: both point into v .

2.3 Adjacency constraint (project default fast path)

The code enforces:

No consecutive nonzero signatures with the same sign.

So $(+2, +2)$ and $(-2, -2)$ adjacent are forbidden, while $(+2, -2)$, $(0, +2)$, etc. are allowed.

3 Symmetry: Why We Need Group Actions

Definition 3.1 (Dihedral group D_n). D_n is the symmetry group of an n -gon. It has:

- n rotations,
- n reflections,

for total size $|D_n| = 2n$.

Definition 3.2 (Action on edge sequences). Your code uses:

- rotations: cyclic shifts of the bit sequence,
- reflections: reverse order *and* flip bits $0 \leftrightarrow 1$.

The bit flip is necessary because reflection reverses geometric direction of each edge.

Definition 3.3 (Equivalent objects). Two sequences are equivalent if one is obtained from the other by some element of D_n .

Definition 3.4 (Orbit). The orbit of an object is all sequences reachable by these symmetries. One orbit = one distinct graph in your count.

4 Canonical Representative

For brute checks, the program computes a canonical form:

1. generate all rotations of the sequence,
2. generate all rotations of reflected-and-bit-flipped sequence,
3. choose lexicographically smallest candidate.

This gives one deterministic representative per orbit.

5 Burnside Lemma: The Orbit Counter

5.1 Statement

Theorem 5.1 (Burnside). For a finite group G acting on finite set X ,

$$\#(X/G) = \frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|,$$

where $\text{Fix}(g) = \{x \in X : g \cdot x = x\}$.

5.2 What “fixed” means

For one symmetry g , an object is *fixed* if applying g leaves it exactly unchanged.

Example: if rotating by 2 steps maps a sequence to itself, that sequence contributes to $|\text{Fix}(g)|$ for that rotation.

5.3 Why this is correct (double-counting sketch)

Count pairs (g, x) with $g \cdot x = x$ in two ways:

- by symmetry first: $\sum_g |\text{Fix}(g)|$,
- by object first: $\sum_x |\text{Stab}(x)|$.

Using orbit-stabilizer, $|\text{Stab}(x)|$ summed over each orbit equals $|G|$, so total is $|G| \times \#\text{orbits}$. Rearranging yields Burnside.

6 Why Burnside Alone Is Not Enough

Burnside tells *how to combine* fixed counts. It does not magically compute those counts.

So we still need fast methods for $|\text{Fix}(g)|$. That is where transfer matrices enter.

7 Transfer Matrix for Adjacency-valid Cycles

7.1 State space used in your code

A local state is:

$$s = (v, e), \quad v \in \{-2, 0, +2\}, \quad e \in \{0, 1\}.$$

So there are 6 states total.

Interpretation:

- e is current edge bit,
- v is signature of the previous vertex.

7.2 One-step update

Given current edge e and next edge e' , the next vertex signature is

$$v' = 2(\mathbf{1}_{e=0} + \mathbf{1}_{e'=1}) - 2.$$

Transition is allowed only if adjacency rule holds between v and v' .

7.3 Matrix construction

Define $M \in \mathbb{Z}_{\geq 0}^{6 \times 6}$ by

$$M_{ij} = \begin{cases} 1, & \text{if transition state } i \rightarrow j \text{ is allowed,} \\ 0, & \text{otherwise.} \end{cases}$$

In your state order

$$(-2, 0), (-2, 1), (0, 0), (0, 1), (2, 0), (2, 1),$$

this matrix is

$$M = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}.$$

7.4 Closed walks and trace

Lemma 7.1. $(M^m)_{ij}$ equals the number of length- m valid state paths from i to j .

Corollary 7.2. The number of valid length- m closed state walks is

$$C(m) = \text{tr}(M^m).$$

This $C(m)$ is the key primitive used in Burnside rotation counts.

8 Burnside Rotation Term in This Project

For a rotation by k , fixed sequences have period $d = \gcd(n, k)$. Grouping rotations by d yields:

$$\text{RotSum}(n) = \sum_{d|n} \varphi\left(\frac{n}{d}\right) C(d),$$

where φ is Euler totient.

Interpretation:

- $C(d)$: how many base periodic blocks of length d satisfy local rules,
- $\varphi(n/d)$: how many rotations have gcd class corresponding to that d .

9 Burnside Reflection Term in This Project

Your code has a separate routine for reflections:

- if n is odd: reflection-fixed count is 0,
- if n is even: compute half-length transfer and enforce reflection boundary compatibility.

Define this code-computed number as $R(n)$.

Then reflection contribution to Burnside sum is:

$$\text{RefTerm}(n) = \frac{n}{2} R(n),$$

matching your implementation.

10 Final Counting Formula in Code

For $n \geq 1$:

$$a(n) = \frac{\text{RotSum}(n) + \text{RefTerm}(n)}{2n} = \frac{1}{2n} \left(\sum_{d|n} \varphi\left(\frac{n}{d}\right) \text{tr}(M^d) + \frac{n}{2} R(n) \right).$$

This is exactly the fast adjacency count route in your cyclic code.

11 Worked Example: $n = 8$ (Exact Values From Your Program)

Divisors of 8 are 1, 2, 4, 8.

d	$\varphi(8/d)$	$C(d) = \text{tr}(M^d)$	contribution
1	4	2	8
2	2	4	8
4	1	16	16
8	1	256	256

So

$$\text{RotSum}(8) = 8 + 8 + 16 + 256 = 288.$$

From reflection routine:

$$R(8) = 16, \quad \text{RefTerm}(8) = \frac{8}{2} \cdot 16 = 64.$$

Final Burnside average:

$$a(8) = \frac{288 + 64}{16} = \frac{352}{16} = 22.$$

So your program returns 22 distinct oriented cyclic graphs (with adjacency rule).

12 Optional Constraints in the Cyclic Module

12.1 Primitivity

A sequence is primitive if it is not a repetition of a shorter block.

Code fast formula uses Mobius inversion:

$$a_{\text{prim}}(n) = \sum_{d|n} \mu(d) a\left(\frac{n}{d}\right),$$

where μ is the Mobius function.

12.2 Forbidden subsequence mode

A signature is rejected if any cyclic contiguous block matches a canonical forbidden pattern learned from smaller lengths. This route uses explicit set-based enumeration (not the adjacency fast matrix shortcut).

13 Validation and Correctness Strategy

Your pipeline validates in layers:

1. Brute-force canonical enumeration for small n .
2. Fast Burnside+matrix counts for adjacency mode.
3. Assert equality on overlapping range.
4. Compare unconstrained small terms with OEIS A053656 values.

This is strong because independent methods agree on the same outputs.

14 Complexity Intuition

Brute force is $O(2^n)$, so it explodes quickly.

Fast path uses:

- divisor loop (depends on number of divisors),
- totient and Mobius on divisor arguments,
- matrix powers (small fixed size matrix, logarithmic exponentiation depth).

Hence very large n is feasible.

15 Presentation-ready 90-second summary

“I model each directed cycle as a binary edge sequence and quotient by dihedral symmetry, so I count true isomorphism classes, not raw strings. Burnside lemma says the number of classes is the average fixed-point count over all symmetries. To compute those fixed counts efficiently, I use a transfer matrix on 6 local states that encode the previous vertex signature and current edge orientation, with adjacency constraint built into allowed transitions. Rotation-fixed counts reduce to weighted trace terms over divisors, and reflection-fixed counts are computed by half-length transfer with boundary compatibility. For example at $n = 8$, rotation sum is 288 and reflection term is 64, giving $(288 + 64)/16 = 22$ distinct classes. Small n is cross-checked by brute-force canonical enumeration and OEIS values.”

16 Glossary of Symbols

Symbol	Meaning
n	cycle length (number of edges and vertices)
e_i	edge orientation bit at edge i , in $\{0, 1\}$
$\sigma(v)$	vertex signature at vertex v , in $\{-2, 0, +2\}$
D_n	dihedral group of order $2n$
G	symmetry group acting on objects
$\text{Fix}(g)$	objects unchanged by symmetry g
M	transfer matrix on 6 local states
$C(m)$	$\text{tr}(M^m)$, number of length- m closed state walks
φ	Euler totient function
μ	Mobius function
$R(n)$	reflection-fixed count computed by reflection routine
$a(n)$	final orbit count (distinct cyclic graphs)

17 Takeaway

Your cyclic module is mathematically clean:

1. define equivalence by dihedral action,
2. compute fixed counts efficiently by transfer matrices,
3. assemble exact orbit counts by Burnside,
4. validate against brute force and known sequences.

That is why the method is both rigorous and scalable.